

智能识别系统设计



群名称:智能识别系统设计2023
群 号:261010657

上课地点: 南区计算机大楼323

深圳大学
计算机与软件学院

● 第二次试验报告

视频人脸检测及识别系统设计

先扣出人脸作为特征提取（规范到同样大小）



1.实验报告主题： 视频人脸检测及识别设计

2.实验目的：

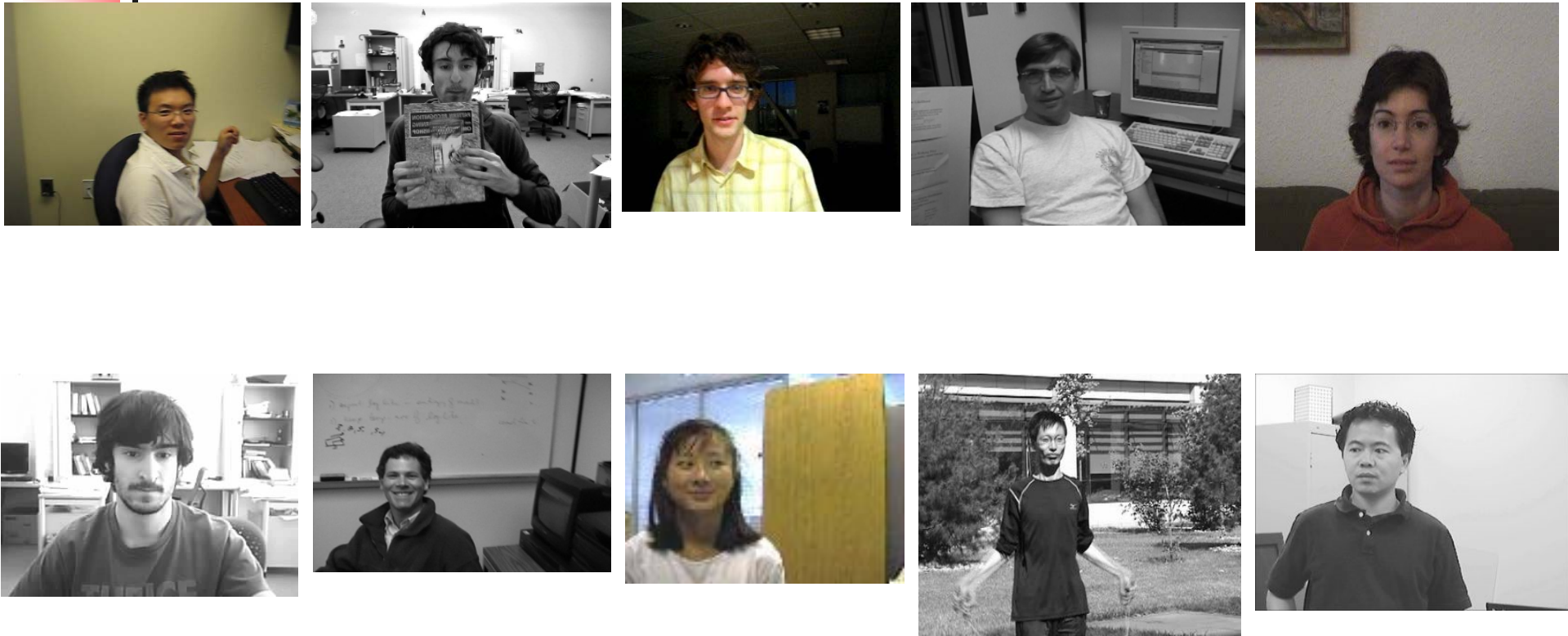
- 1) 编程读取人脸视频或图像序列并实现对人脸的检测与预处理（提示：人脸检测方法可采用**AdaBoost**、人脸关键点提取等机器学习方法）；
- 2) 编写**PCA**或其他特征提取算法对人脸图像进行特征提取；
- 3) 设计并构建人脸识别框架。

训练数据集的处理

3.实验要求：

- 1) 熟悉**OpenCV**编程环境或其他图像处理工具包；
- 2) 熟练掌握基本图像处理方法和**PCA**子空间学习等人脸特征提取方法；
- 3) 熟悉最近邻等图像分类方法；
- 4) 编程语言推荐使用**C++**，但不限于**C++**（例如可使用**Python**或**Matlab**语言）；
- 5) 无需编写界面，代码写成函数，方便调用（若以**C++**为编程语言，建议以类的形式把相关函数统一起来，方便后续使用）。

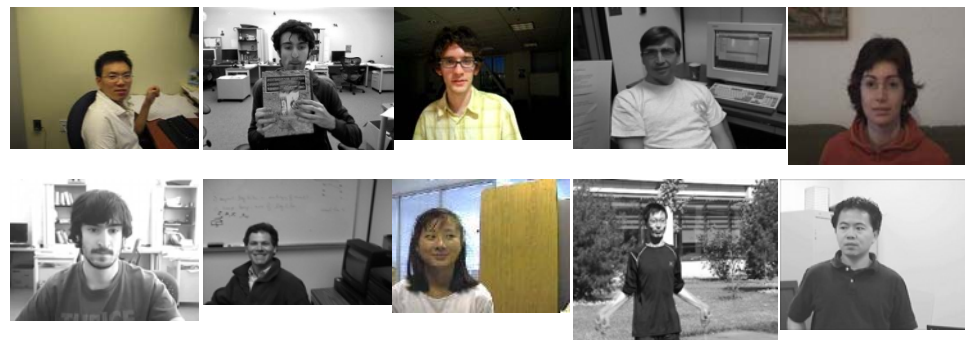
● 视频人脸数据库介绍



共**10**个类别，每个类在**train**文件夹中选**10**张作为训练对象

● 第二次试验报告

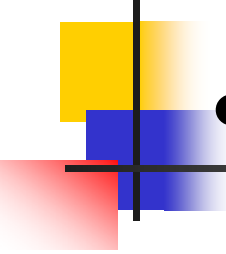
视频人脸检测及识别系统设计



实验报告内容：

- 1) 从视频或图像序列中创建训练样本，其中类别个数为**10**，每类样本有**10**个人脸图像；
- 2) 采用**PCA**或其他人脸特征提取方法提取人脸特征，并存储到文本文件中（提示：若用**PCA**方法，则需要把训练样本投影到子空间，获得训练样本在子空间的投影以及投影矩阵），以便测试时调用；
- 3) 对**10**个类别的测试视频或测试图像序列进行测试，对于每一个类，输入其测试视频或测试图像序列，然后通过人脸检测、归一化、分类等一系列方法实现对视频或图像序列中人脸的识别，并统计识别率，若**10**个测试视频（或图像序列）都识别正确，则识别率为**100%**。

提交时间：2023年11月4日下午5:00



● 本次课程任务

1. 课程回顾
2. 部分练习题解疑
3. 视频分析基础
 - (1) 视频特点及难点
 - (2) 从视频流的产生到后端处理
 - (3) 视频中的特征提取
4. OpenCV中的基本视频操作
5. 运动目标检测
6. 图像和视频中的人脸检测
7. 运动目标跟踪
8. 完成课堂编程任务

● 课程回顾



- OpenCV入门及编程环境配置 → 编程要求：类+STL模板库
- 图像读取、遍历、感兴趣区域提取
- 图像去噪及边缘提取：滤波算法 → 结合多方向算子的边缘检测
- 单通道图像的二值化方法
- 图像中的轮廓提取
- 图像特征提取 → LBP、Gabor、颜色直方图
- 实例分析 → (1) 基于颜色直方图目标查找； (2) 指尖检测等

● 部分练习题解疑

关于Sobel算子图像边缘提取:

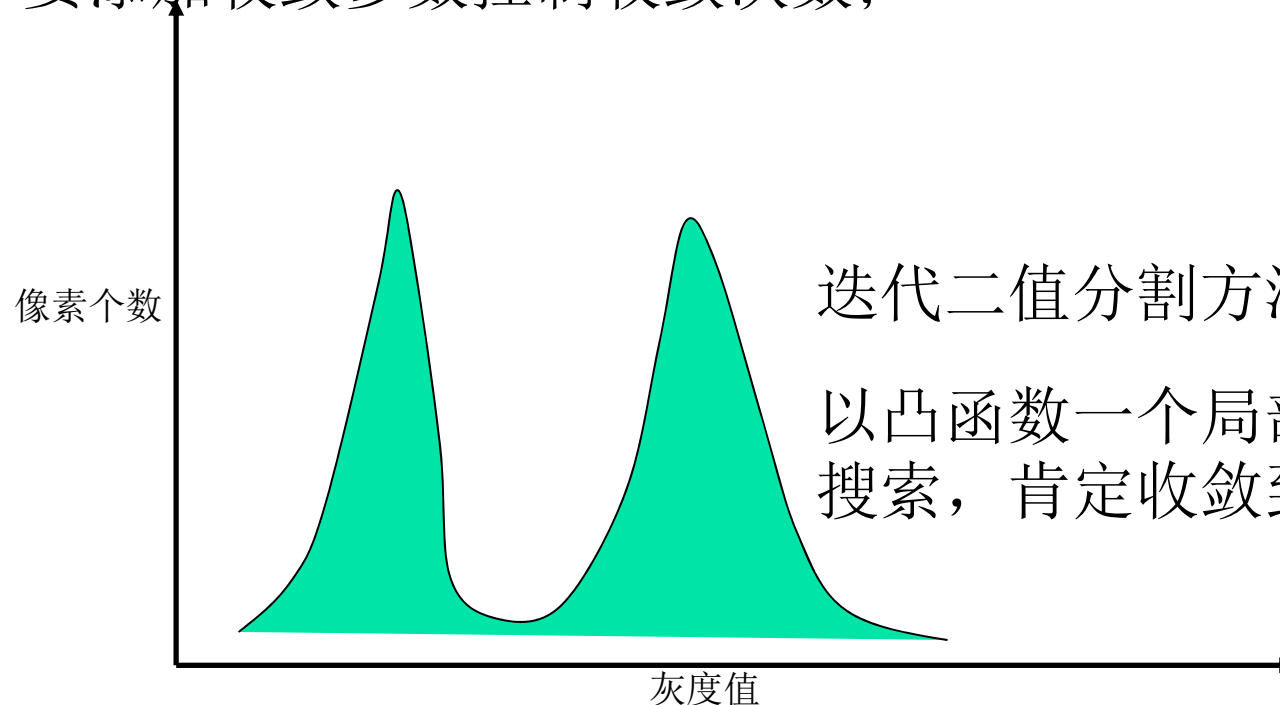
- 1) 边界不处理;
- 2) 扩大2个像素, 创建更大的图像保存结果;
- 3) 通过约束条件的方式处理边界点

```
Mat_<uchar> dataMat=InputImg;  
Mat_<float> dataResultMat=Gray32Mat;  
int oh=OperatorSize/2;//算子Operator的半径  
int ow=OperatorSize/2;  
for(int j=0;j<InputImg.rows;j++)  
{  
    for(int i=0;i<InputImg.cols;i++)  
    {  
        float datavalue=0;  
        for(int p=-oh;p<=oh;p++)  
        {  
            for(int q=-ow;q<=ow;q++)  
            {  
                if(p+j<0||p+j>InputImg.rows-1||q+i<0||q+i>InputImg.cols-1)  
                {  
                    continue;  
                }  
                datavalue+=dataMat(j+p,i+q)*Operator[(p+oh)*OperatorSize+(q+ow)];  
            }  
        }  
        dataResultMat(j,i)=saturate_cast<float>(datavalue);  
    }  
}
```

● 部分练习题解疑

动态阈值分割:

- 1) 每次更新阈值, 都直接在原图统计大于某个阈值的像素个数, 效率很低;
- 2) 灰度图计算错误, 统计的是像素个数, 不是像素值累加;
- 3) 要添加收敛参数控制收敛次数;



迭代二值分割方法的假设

以凸函数一个局部最优解开始
搜索, 肯定收敛到全局最优解。

- 补充内容：自动阈值分割法—迭代二分法（使用迭代法求最佳阈值T）

Step1: 求图像的最大、最小灰度值 T_{max} 和 T_{min} ，令初始阈值为：

$$T_0 = \frac{T_{max} + T_{min}}{2}$$

Step2: 根据阈值 T_k ，将图像分割成目标和背景两部分，求出两部分的平均灰度：

$$Z_F = \frac{\sum_{Z(i,j) < T_k} Z(i,j) \times N(i,j)}{\sum_{Z(i,j) < T_k} N(i,j)}$$

$$Z_B = \frac{\sum_{Z(i,j) > T_k} Z(i,j) \times N(i,j)}{\sum_{Z(i,j) > T_k} N(i,j)}$$

灰度值

像素点
个数

Step3: 求出新阈值

$$T_{K+1} = \frac{Z_F + Z_B}{2}$$

如果 $|T_{K+1} - T_K| < \varepsilon$ 或者迭代次数大于规定的次数，则程序结束

否则 $K \leftarrow K + 1$ 转Step2。

● 视频分析基础

(1) 视频特点及难点



来自图像空域（表观）的挑战
（噪声、相似背景、光照）
车身和背景相同颜色



随时间变化带来的挑战
（遮挡、姿态变化）

时空两方面挑战

● 视频分析基础

(1) 视频特点及难点



光照干扰



目标姿态变化 及 目标之间遮挡干扰



动态背景干扰



噪声模糊



抖动干扰

● 视频分析基础

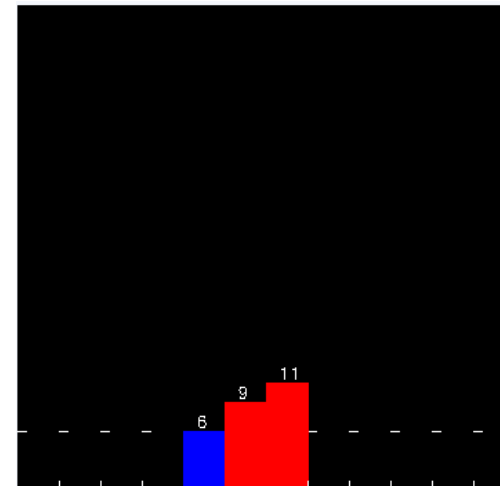
(3) 视频中的特征提取

角点特征

```
cvFindCornerSubPix(imgA, //image:输入图像
cornersA, //corners:输入角点的初始坐标, 也存储精确的输出坐标
corner_count, //corner:角点数目
cvSize(win_size, win_size), //win:搜索窗口的一半尺寸。如果win=(5,5), 那么所用搜索窗口的大小为(5*2+1)*(5*2+1)=11*11
cvSize(-1,-1), //死区的一半尺寸。死区为不对搜索区中央位置做求和运算的区域。它用来避免自相关矩阵出现
//某些可能的奇异性。当值为(-1,-1)时表示没有死区
cvTermCriteria(CV_TERMCRIT_ITER|CV_TERMCRIT_EPS, 20, 0.03) ); //求角点迭代过程的终止条件, 即
//对于角点位置的确定, 或者迭代数大于某个设定值, 或者精确度达到某个设定值。criteria可以是
//最大迭代数, 也可以是设定的精确度, 还可以是他们的组合
```

光流特征

```
cvCalcOpticalFlowPyrLK(imgA, //prev:在时间t的第一帧
imgB, //curr:在时间t+dt的第二帧
pyrA, //prev_pyr:第一帧的金字塔缓存。如果指针非NULL, 则缓存必须有足够的空间来存储金字塔从
//层1到层level的内容。缓存大小至少为(image_width+8)*image_height/3字节
pyrB, //与prev_pyr类似, 用于第二帧
cornersA, //需要发现光流的点集
cornersB, //包含新计算出来的位置的点集
corner_count, //特征点数
cvSize(win_size, win_size), //每个金字塔层的搜索窗口尺寸
5, //最大的金字塔层数。如果为0, 不使用金字塔(即金字塔为单层); 如果为1, 使用两层, 依次类推
features_found, //status:状态数组。如果对应特征的光流被发现, 则数组中的每一个元素都被置1; 否则置0
feature_errors, //双精度数组, 包含原始图像碎片与移动点之间的差。可选参数, 可以为NULL
cvTermCriteria(CV_TERMCRIT_ITER|CV_TERMCRIT_EPS, 20, 3), //准则, 指定在每个金字塔层, 为某点寻找光流迭代过程的终止条件
0); //其他选项, 为下列值之一:
```



● OpenCV中的基本视频操作

VideoCapture类

open (const **String** &filename) //打开视频
isOpened () const //检测打开是否成功

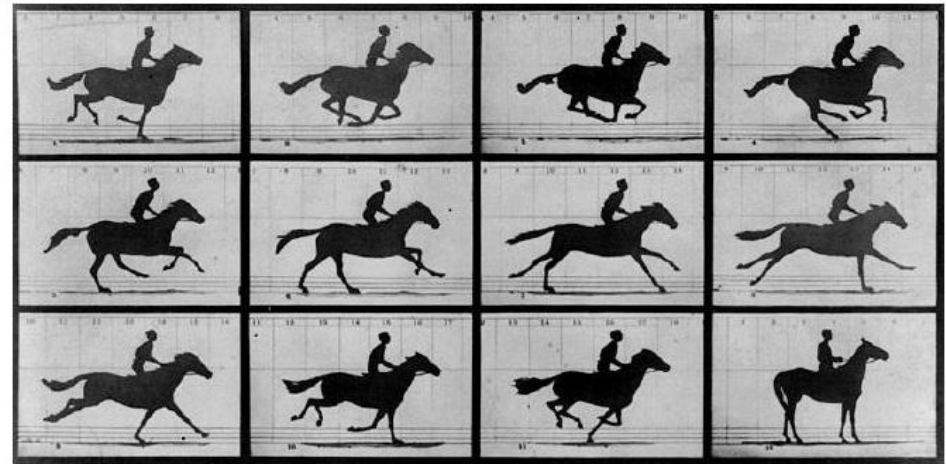
get (int propId) const //视频属性

operator>> (**Mat** &image) //取帧
read (**OutputArray** image)

grab () //缓存后取帧
retrieve (**OutputArray** image, int flag=0);

release () //释放文件

- CAP_PROP_POS_MSEC Current position of the video file in milliseconds or video capture timestamp.
- CAP_PROP_POS_FRAMES 0-based index of the frame to be decoded/captured next.
- CAP_PROP_POS_AVI_RATIO Relative position of the video file: 0 - start of the film, 1 - end of the film.
- CAP_PROP_FRAME_WIDTH Width of the frames in the video stream.
- CAP_PROP_FRAME_HEIGHT Height of the frames in the video stream.
- CAP_PROP_FPS Frame rate.
- CAP_PROP_FOURCC 4-character code of codec.
- CAP_PROP_FRAME_COUNT Number of frames in the video file.
- CAP_PROP_FORMAT Format of the **Mat** objects returned by retrieve() .
- CAP_PROP_MODE Backend-specific value indicating the current capture mode.



- OpenCV中的基本视频操作

VideoWriter类

open (const **String** &filename, int **fourcc**, double fps, **Size** frameSize, bool isColor=true) //存储视频文件信息

VideoWriter::fourcc //-1弹窗口自选

压缩编码规则, <http://www.fourcc.org/codecs.php>

set (int propId, double value) //设置属性

operator<< (const **Mat** &image) //写图片

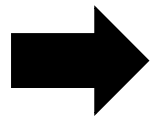
write (const **Mat** &image)

release () //释放文件

● OpenCV中的基本视频操作

- ① 变量声明: VideoCapture capture
- ② 打开视频: capture.open(string filename)
- ③ 读取视频帧: capture>>fame, 其中fame是Mat变量
- ④ 判断视频是否已打开: capture.isOpened()
- ⑤ 判断帧是否为空: if(fame.empty())

```
string filename = "face_test.avi"; //打开的视频文件
VideoCapture capture;
capture.open(filename);
double rate = capture.get(CV_CAP_PROP_FPS); //获取视频文件帧率
int delay = cvRound(1000.000 / rate);
if (!capture.isOpened()) //判断是否打开视频文件
{
    printf("离开\n");
    exit(0);
}
else
{
    while (true)
    {
        Mat frame;
        capture >> frame; //读出每一帧的图像
        if (frame.empty()) break;
        imshow("处理前视频", frame);
        //processImage(frame);
        imshow("处理后视频", frame);
        waitKey(delay);
    }
}
```



视频读取关键代码

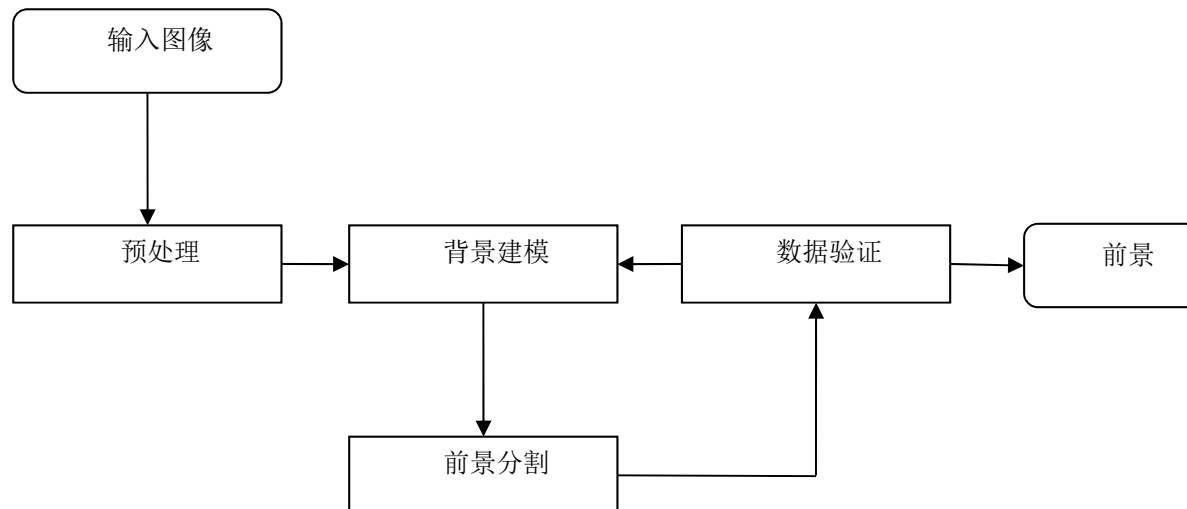
- ① 摄像头变量声明: VideoCapture capture(0)
- ② 摄像头是否成功打开: capture.isOpened()
- ③ 摄像头图像读取: capture.read(frame), 其中fame是Mat变量

```
VideoCapture capture(0);
if (!capture.isOpened()) //检查摄像头是否成功打开
{
    printf("视频没有读取成功\n");
    exit(0);
}
//这里必须两个一起设置才能生效, 宽和高
capture.set(CV_CAP_PROP_FRAME_WIDTH, 1920); //1080;
capture.set(CV_CAP_PROP_FRAME_HEIGHT, 1080); //1080;
//capture.set(CV_CAP_PROP_SHARPNESS, 100); //1080;
Mat frame; //当前视频帧
int i = 0;
while(1)
{
    if (!capture.read(frame))
    {
        break;
    }
    imshow("video", frame);
    waitKey(30);
    if (i++ > 100)
    {
        break;
    }
}
```

摄像头读取关键代码

●运动目标检测

前景检测基本原理



- 运动目标检测

视频目标=阈值化（当前图像帧-背景图像帧）



 ●运动目标检测

平均背景建模

选取一定帧数的图像出来进行求和再求平均作为背景

平均式:

$$B(t) = \frac{1}{n} \sum_{t=1}^n I(t)$$

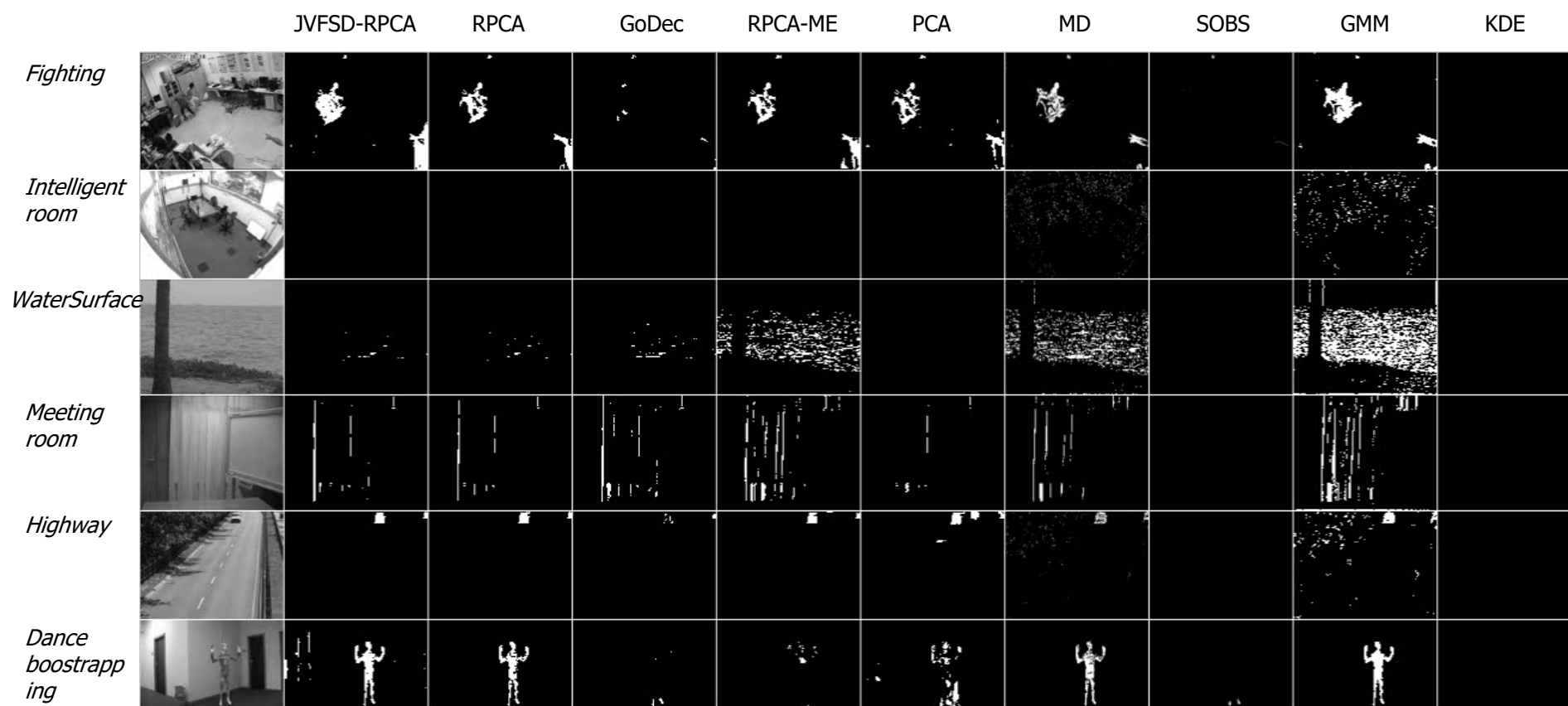
平滑式:

$$B(t) = \alpha B(t-1) + (1-\alpha) I(t), \quad t = 1, 2, \dots, n$$

后续背景根据加权赋值进行改变更新

$$0 < \alpha < 1$$

● 运动目标检测



我们方法

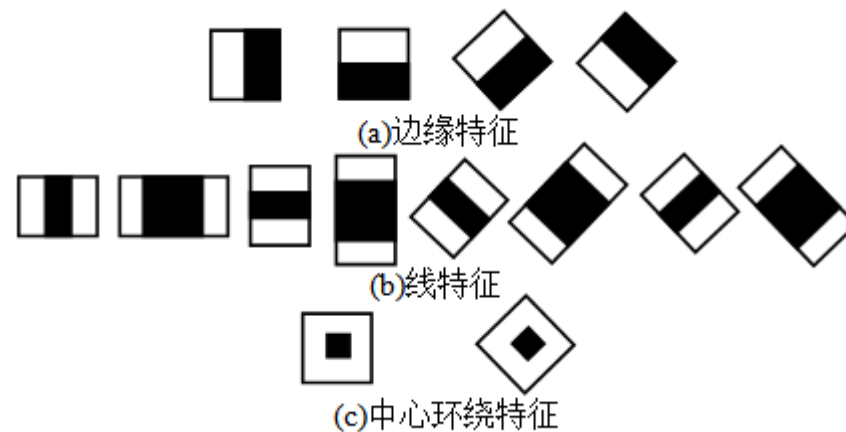
实验结果

● 图像和视频中的人脸检测

1. AdaBoost算法

- a) Rainer L, Jochen M. An Extended Set of Haar-like Features for Rapid Object Detection [C]//Proceedings of 16th IEEE International Conference on Image Processing. Piscataway, NJ, USA: IEEE Press, 2002: 900-903.
- b) Paul V, Michael Jones. Rapid Object Detection using a Boosted Cascade of Simple Features [C]//Proceedings of the IEEE Computer Vision and Pattern Recognition. Piscataway, NJ, USA: IEEE Press, 2001: 151-155.

2. Haar特征



● 图像和视频中的人脸检测

1. 变量和关键函数

- ① 变量声明: `CascadeClassifier cascade_faceDetector;`
- ② 人脸检测: `cascade_faceDetector.detectMultiScale(const Mat& image,
CV_OUT vector<Rect>& objects,
double scaleFactor=1.1,
int minNeighbors=3, int flags=0,
Size minSize=Size(),
Size maxSize=Size())`

2. OpenCV对应的库函数

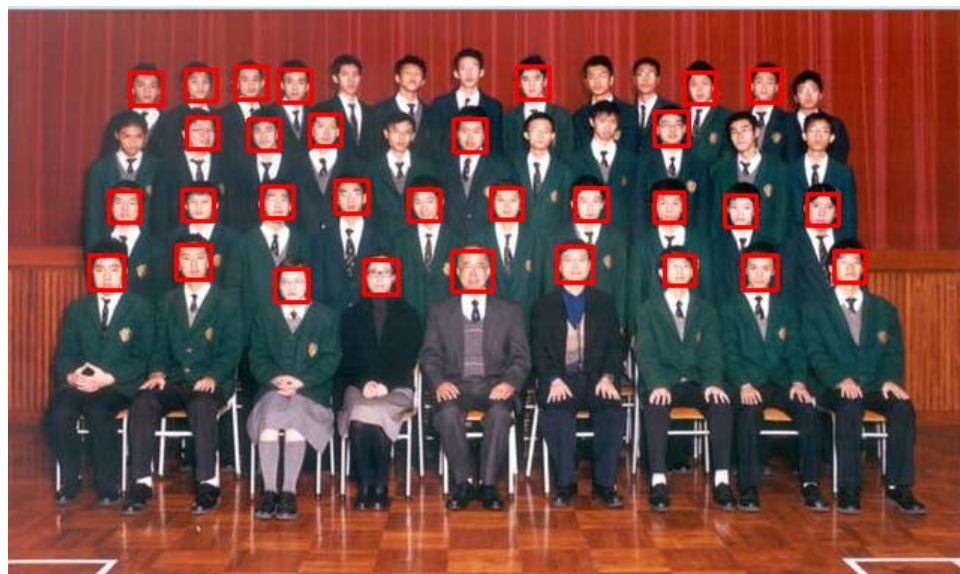
需要的库函数 `opencv_objdetect249d.lib`

3. 创建人脸检测的类

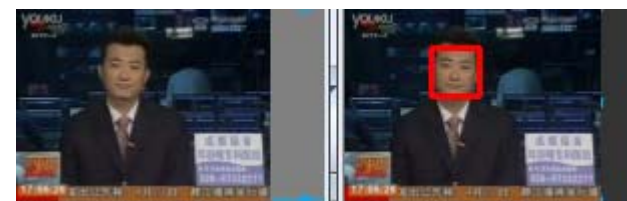
```
#ifndef _HAARDETECTION_H_
#define _HAARDETECTION_H_
//opencv_objdetect247d.lib
#include <vector>
#include <opencv2/opencv.hpp>
using namespace std;
using namespace cv;
class CHaarDetection
{
public:
    CascadeClassifier cascade_faceDetector;
public:
    CHaarDetection(void);
    ~CHaarDetection(void);

    //最多返回10个人脸
    //在图像中的局部区域LocalArea进行Adaboost, 并返回全局坐标的人脸, 而不是局部区域上的位置
    void FaceDetection(Mat image, Rect LocalArea, vector<Rect>& FaceVector, vector<Rect>& PreProcessFaceVector);
    void DrawOnImg(Mat &image, vector<Rect>& Faces);
    void TestOnImgDemo();
    void TestOnVideoDemo();
};
#endif
```

● 图像和视频中的人脸检测

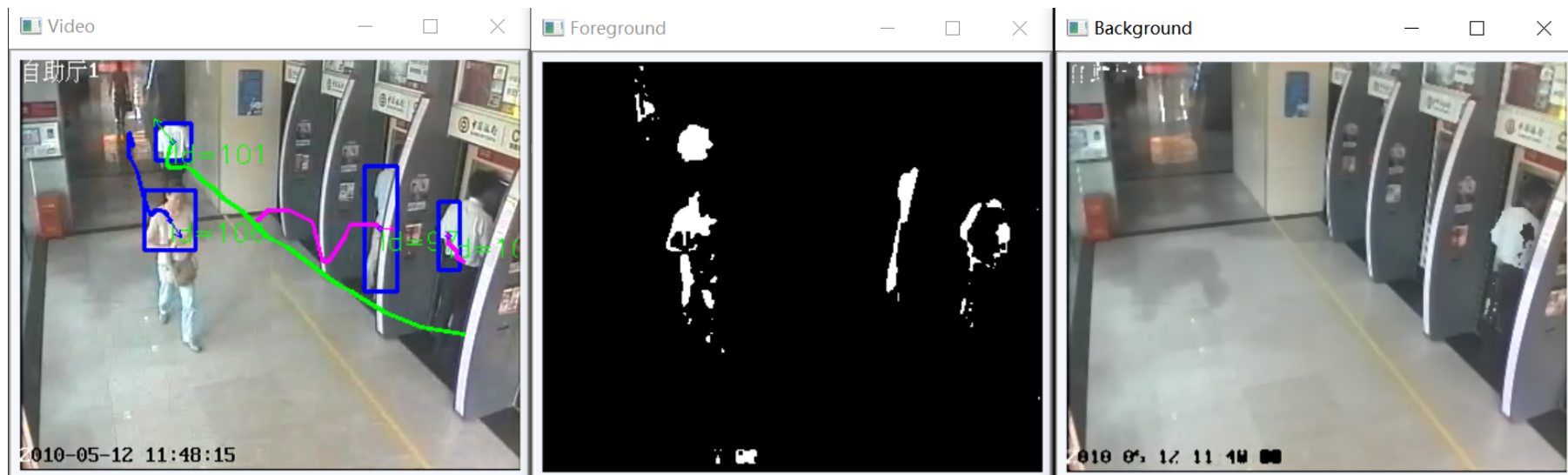


图像中人脸检测

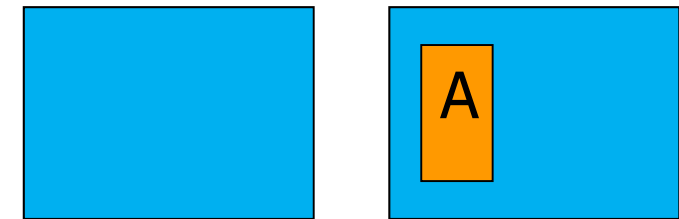
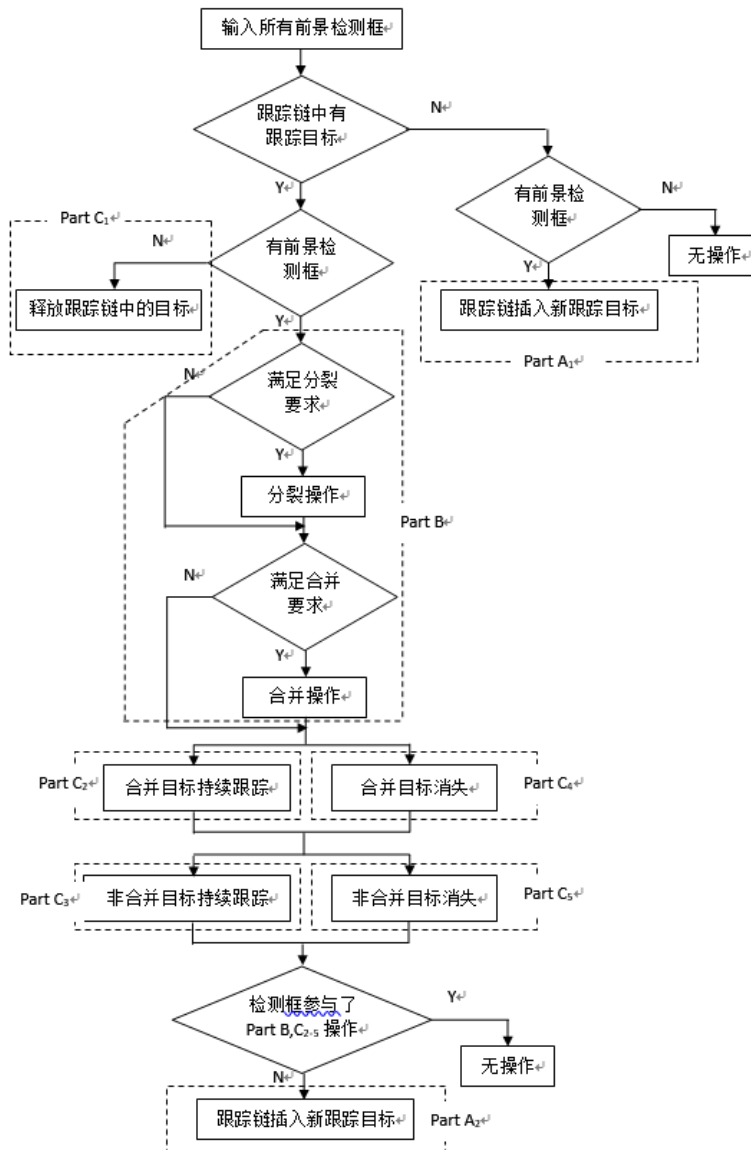


视频中人脸检测

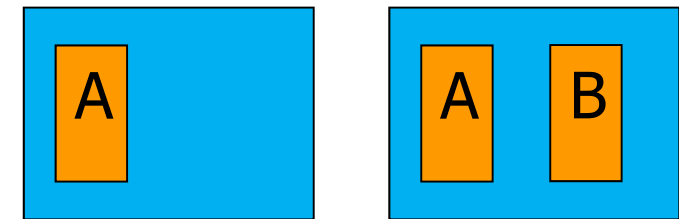
● 运动目标跟踪



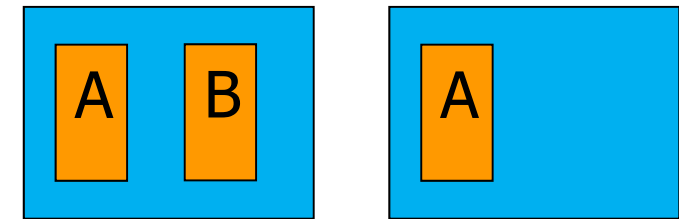
●运动目标跟踪



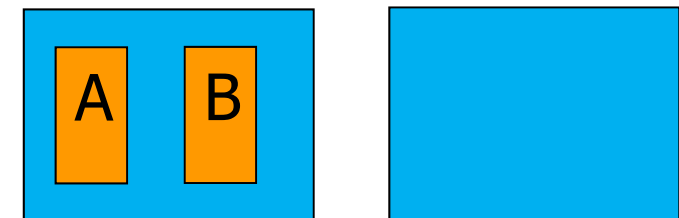
(1)



(2)

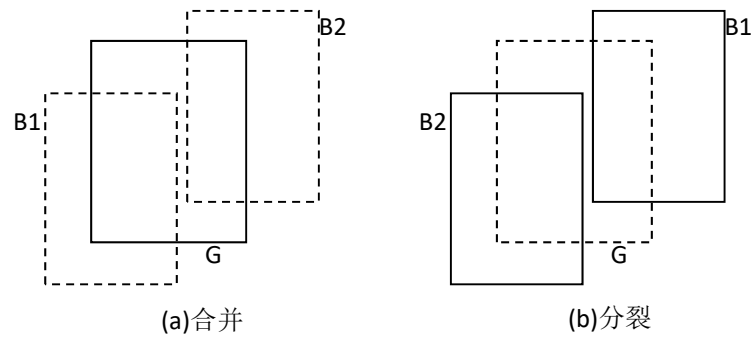


(3)



(4)

● 运动目标跟踪



目标的分裂与合并

● 视频分析基础

基于运动目标检测可以做什么：

